

DML

(SELECT-2)

• ORDER BY

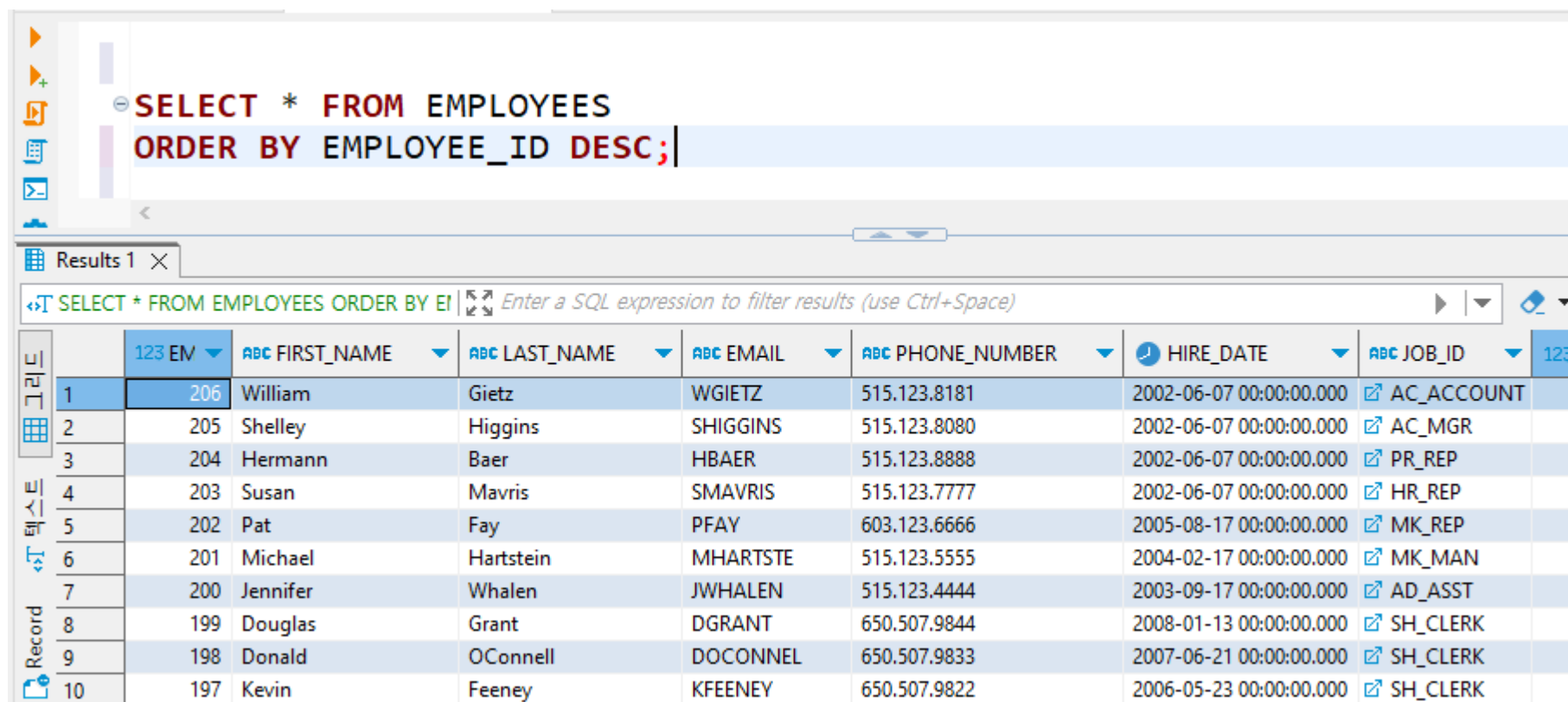
SELECT에 의해서 조회된 데이터를 정렬해주는 구문으로써, 데이터의 조회 후 실행되기 때문에 SELECT문의 가장 마지막에 실행됨

아래의 옵션은 생략 가능함

옵션	설명
(default) ASC	오름차순
DESC	내림차순
NULLS FIRST	컬럼값에 NULL이 존재할 경우 NULL을 앞에 정렬시킴 (DESC를 사용할 경우 기본값)
NULLS LAST	컬럼값에 NULL이 존재할 경우 NULL을 뒤에 정렬시킴 (ASC를 사용할 경우 기본값)

- ORDER BY

EMPLOYEE_ID 컬럼을 내림차순으로 정렬

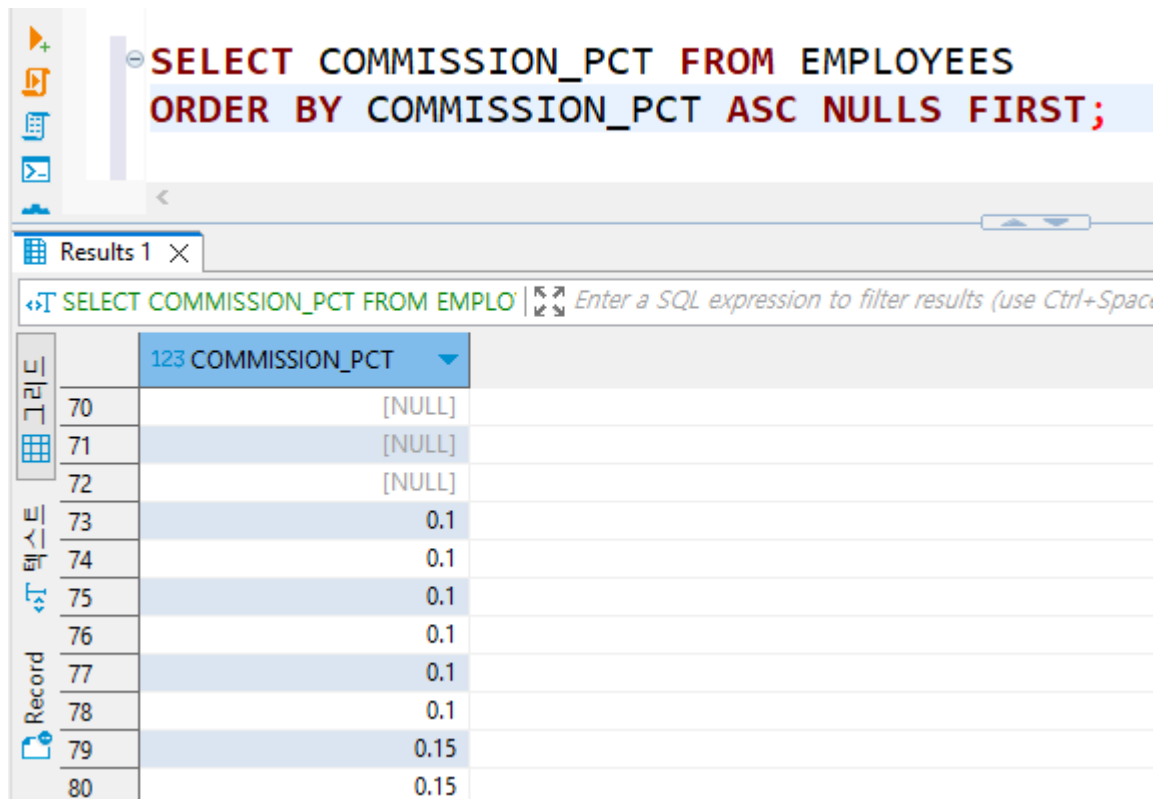


The screenshot shows a SQL IDE interface. At the top, a query editor contains the SQL statement: `SELECT * FROM EMPLOYEES ORDER BY EMPLOYEE_ID DESC;`. Below the editor, a tab labeled 'Results 1' is active, displaying the query results in a table. The table has 8 columns: EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, HIRE_DATE, and JOB_ID. The results are ordered by EMPLOYEE_ID in descending order, from 206 down to 197. The table includes 10 rows of data, each with a row number in the first column.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
1	206	William	Gietz	WGIEZT	515.123.8181	2002-06-07 00:00:00.000	AC_ACCOUNT
2	205	Shelley	Higgins	SHIGGINS	515.123.8080	2002-06-07 00:00:00.000	AC_MGR
3	204	Hermann	Baer	HBAER	515.123.8888	2002-06-07 00:00:00.000	PR_REP
4	203	Susan	Mavris	SMAVRIS	515.123.7777	2002-06-07 00:00:00.000	HR_REP
5	202	Pat	Fay	PFAY	603.123.6666	2005-08-17 00:00:00.000	MK_REP
6	201	Michael	Hartstein	MHARTSTE	515.123.5555	2004-02-17 00:00:00.000	MK_MAN
7	200	Jennifer	Whalen	JWHALEN	515.123.4444	2003-09-17 00:00:00.000	AD_ASST
8	199	Douglas	Grant	DGRANT	650.507.9844	2008-01-13 00:00:00.000	SH_CLERK
9	198	Donald	OConnell	DOCONNEL	650.507.9833	2007-06-21 00:00:00.000	SH_CLERK
10	197	Kevin	Feeney	KFEENEY	650.507.9822	2006-05-23 00:00:00.000	SH_CLERK

- ORDER BY

EMPLOYEE_ID 컬럼을 오름차순으로 정렬하되, NULL 값을 앞에 정렬



The screenshot shows a SQL IDE interface. At the top, a query is entered in a text area: `SELECT COMMISSION_PCT FROM EMPLOYEES ORDER BY COMMISSION_PCT ASC NULLS FIRST;`. Below the query, a tab labeled 'Results 1' is active. The results are displayed in a table with two columns: 'EMPLOYEE_ID' and 'COMMISSION_PCT'. The table shows 11 rows of data, sorted by commission percentage in ascending order, with NULL values appearing first. The interface includes a toolbar on the left with icons for query execution and a status bar at the bottom.

EMPLOYEE_ID	COMMISSION_PCT
70	[NULL]
71	[NULL]
72	[NULL]
73	0.1
74	0.1
75	0.1
76	0.1
77	0.1
78	0.1
79	0.15
80	0.15

• 그룹 함수

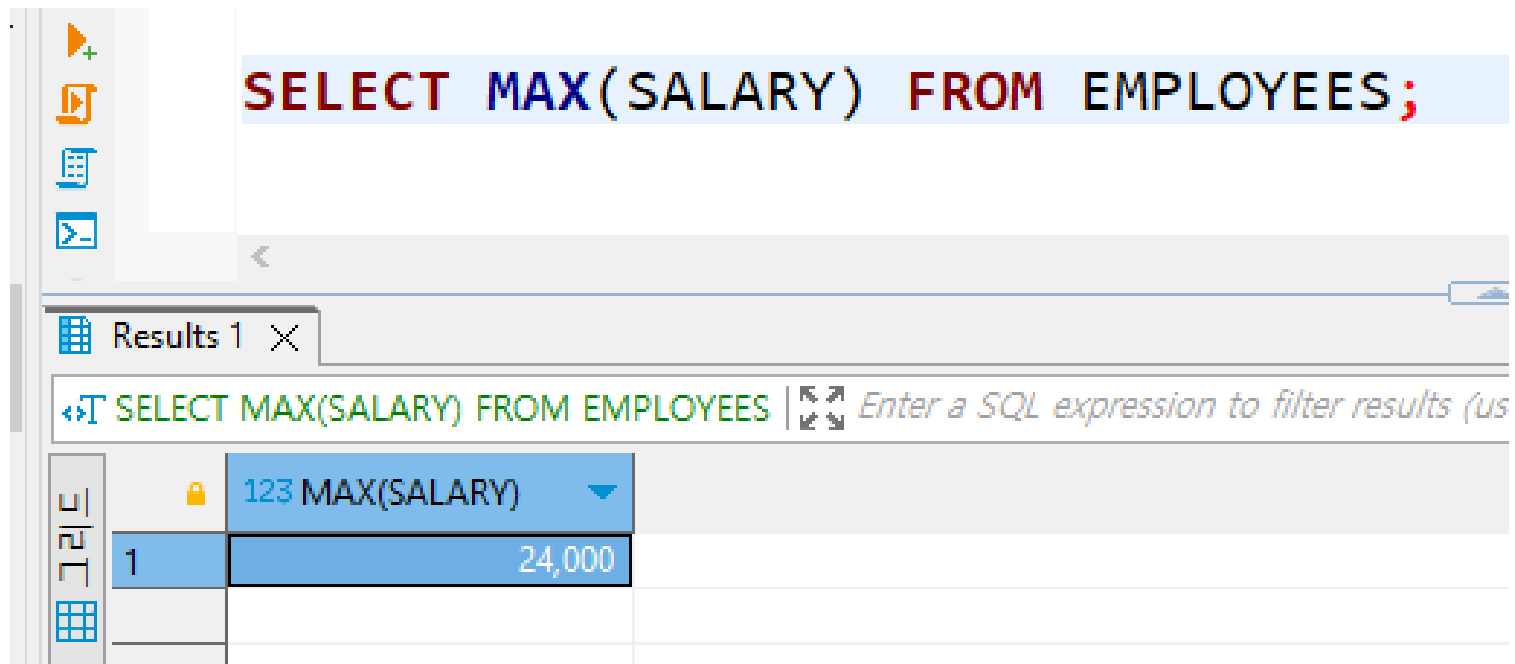
여러 개의 행을 묶어 연산하여 결과를 반환하며 자주 쓰이는 그룹 함수는 아래와 같음

* Oracle의 내장함수 중 한가지 분류임. 그룹함수 외에도 문자함수, 수치 함수, 날짜 함수 등의 내장 함수들이 존재함

그룹 함수	설명
MAX	최대 값
MIN	최소 값
AVG	평균
SUM	합계
COUNT	반환된 행의 개수

- 그룹함수 - MAX

EMPLOYEES 테이블에 있는 SALARY 컬럼의 최대 값을 반환

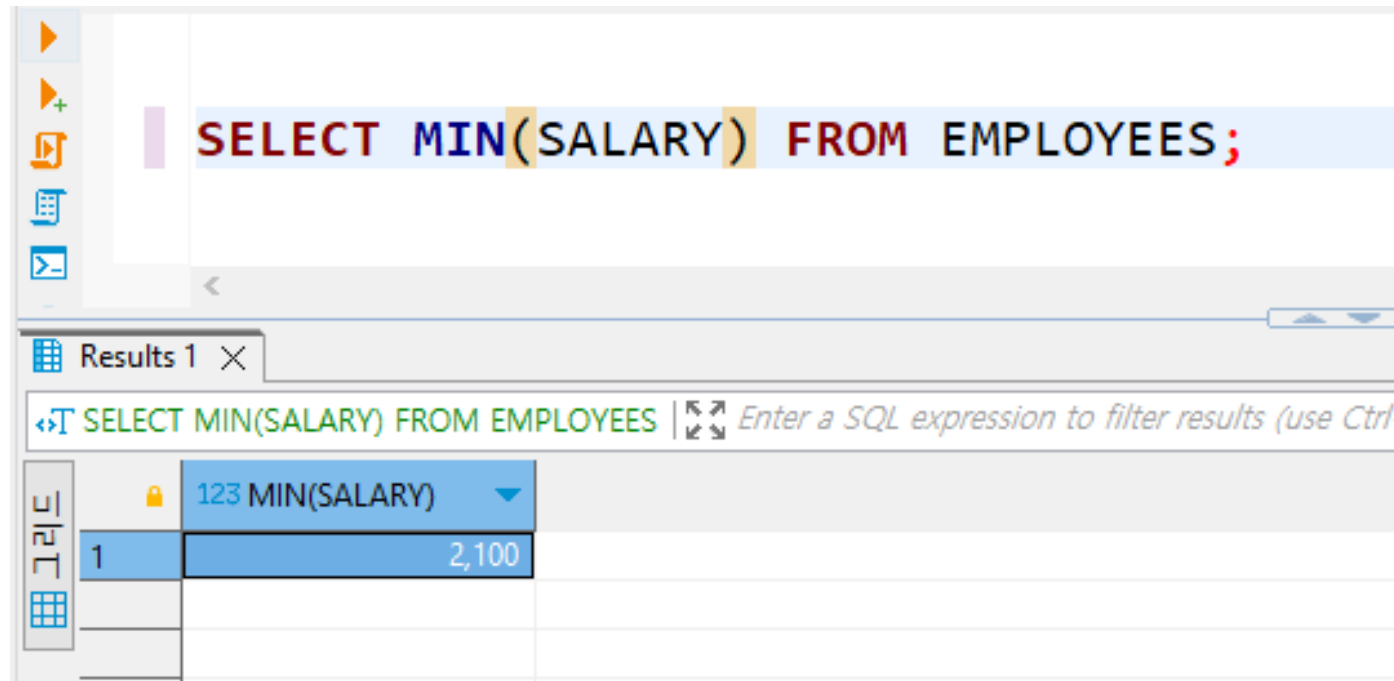


The screenshot displays a SQL development environment. At the top, a SQL query is entered in a text area: `SELECT MAX(SALARY) FROM EMPLOYEES;`. Below the query editor, a tab labeled 'Results 1' is active, showing the query result in a table format. The table has one column, '123 MAX(SALARY)', and one row with the value '24,000'. The interface includes a toolbar on the left with icons for query execution and a status bar at the bottom.

	123 MAX(SALARY)
1	24,000

- 그룹함수 - MIN

EMPLOYEES 테이블에 있는 SALARY 컬럼의 최소 값을 반환

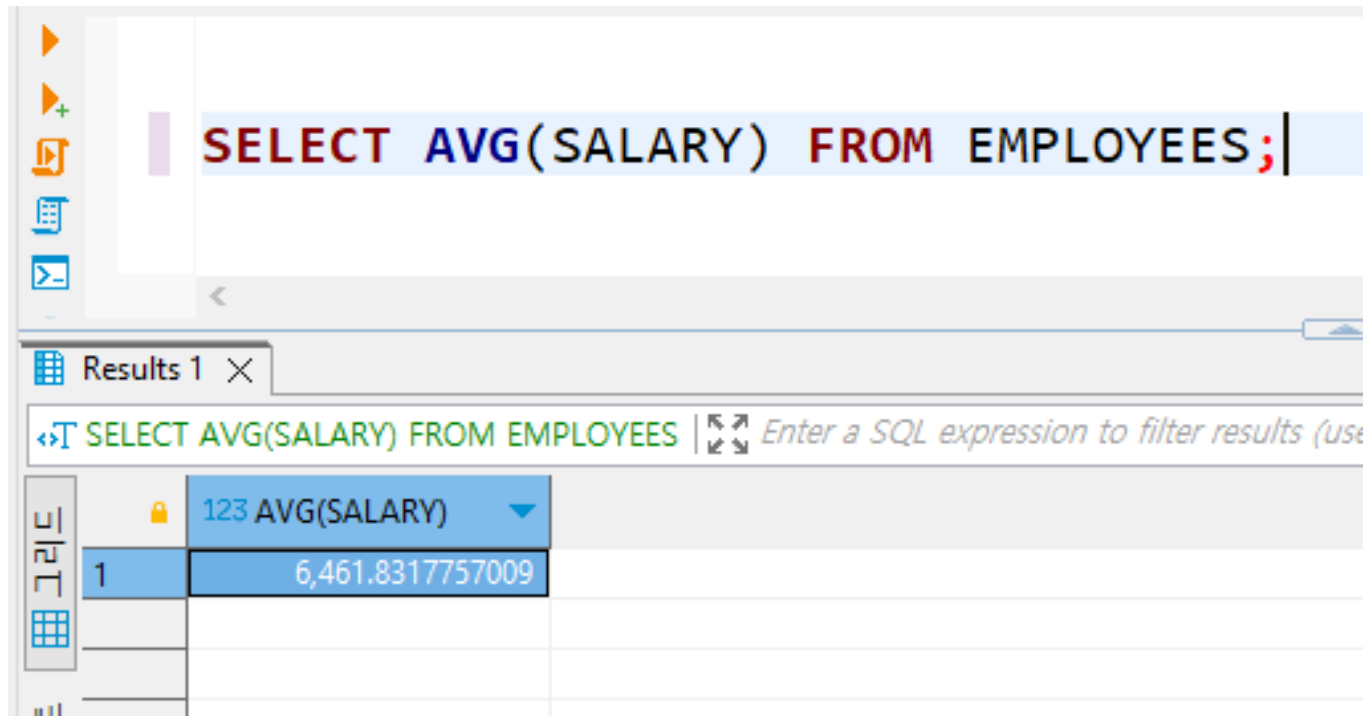


The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT MIN(SALARY) FROM EMPLOYEES;`. The bottom pane, titled 'Results 1', shows the execution result. It includes a SQL statement editor with `SELECT MIN(SALARY) FROM EMPLOYEES` and a results grid. The grid has a single row with the value 2,100.

	123 MIN(SALARY)
1	2,100

- 그룹함수 - AVG

EMPLOYEES 테이블에 있는 SALARY 컬럼의 평균 값을 반환

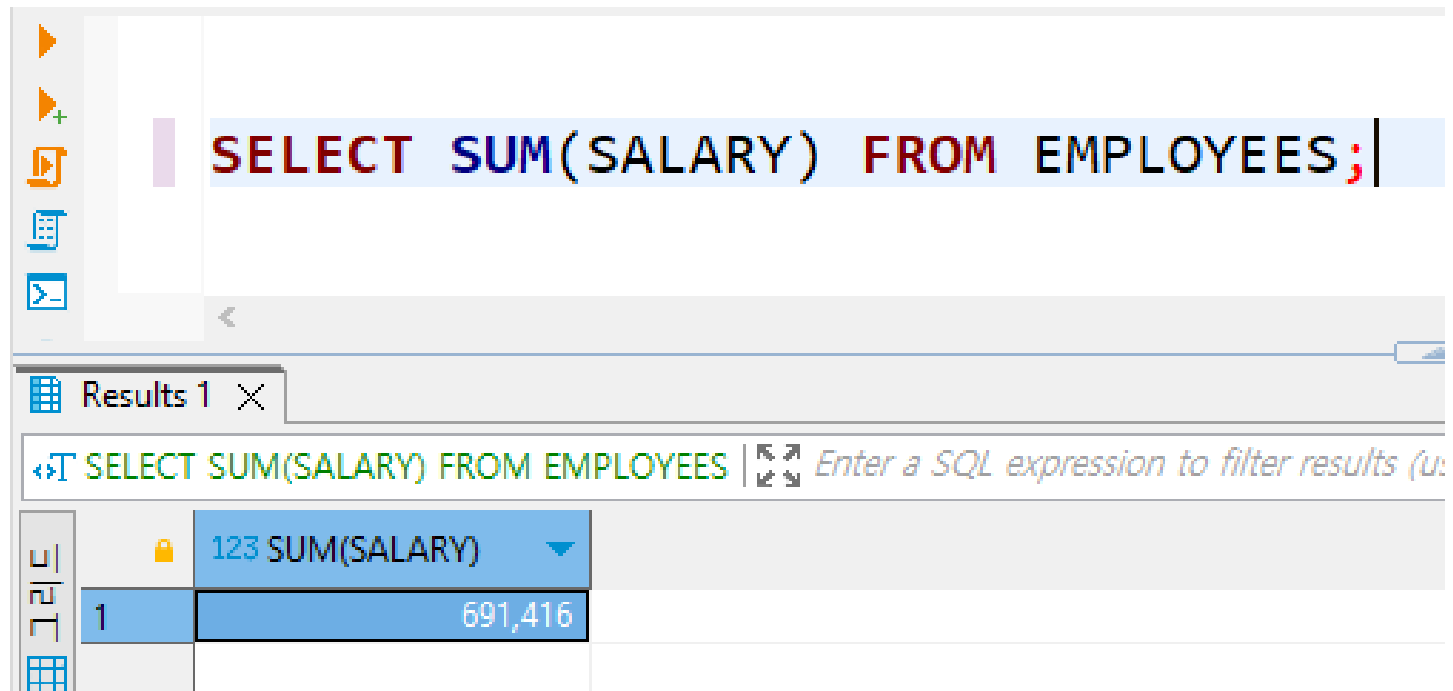


The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT AVG(SALARY) FROM EMPLOYEES;`. The bottom pane, titled 'Results 1', shows the query result in a table format. The table has one column, 'AVG(SALARY)', and one row with the value '6,461.8317757009'.

	123 AVG(SALARY)
1	6,461.8317757009

- 그룹함수 - SUM

EMPLOYEES 테이블에 있는 SALARY 컬럼의 합계 값을 반환



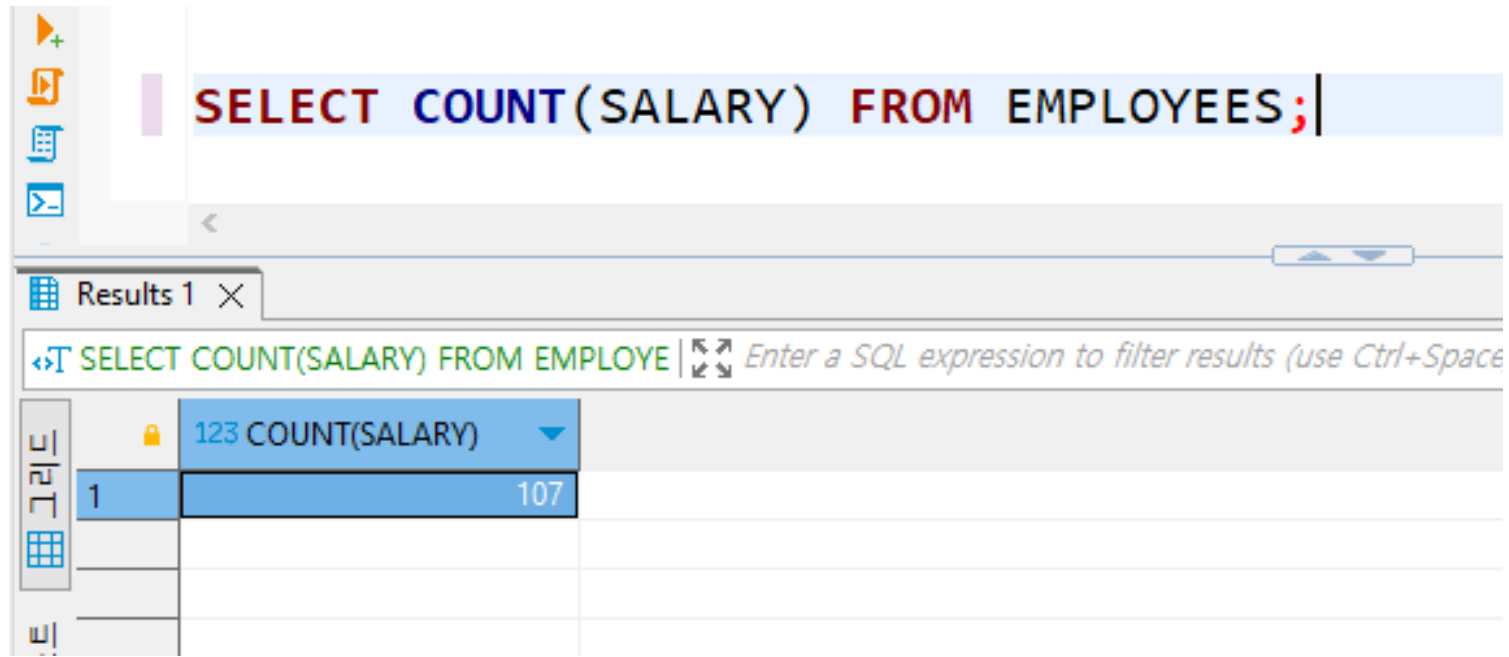
The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `SELECT SUM(SALARY) FROM EMPLOYEES;`. The bottom pane shows the results of the query in a table format. The table has one column, `SUM(SALARY)`, and one row with the value `691,416`.

	123 SUM(SALARY)
1	691,416

DML

• 그룹함수 - COUNT

EMPLOYEES 테이블에 있는 SALARY 컬럼의 데이터가 조회된 행의 개수를 반환



The screenshot shows a SQL IDE interface. At the top, a query editor contains the SQL statement: `SELECT COUNT(SALARY) FROM EMPLOYEES;`. Below the editor, a results pane titled "Results 1" displays the output of the query. The results are shown in a table with two columns: a row identifier and the count value. The first row shows a count of 107.

	123 COUNT(SALARY)
1	107

• Q1. 실습

1. DEPARTMENT_ID가 80인 직원들의 월급 합계를 구하세요.
2. JOB_ID가 IT_PROG인 직원들의 수를 구하세요.
3. HIRE_DATE가 2006년 이후인 직원들의 평균 월급을 구하세요.
4. 직원들 중 가장 월급이 낮은 값과 가장 월급이 큰 값을 구하세요.

• GROUP BY

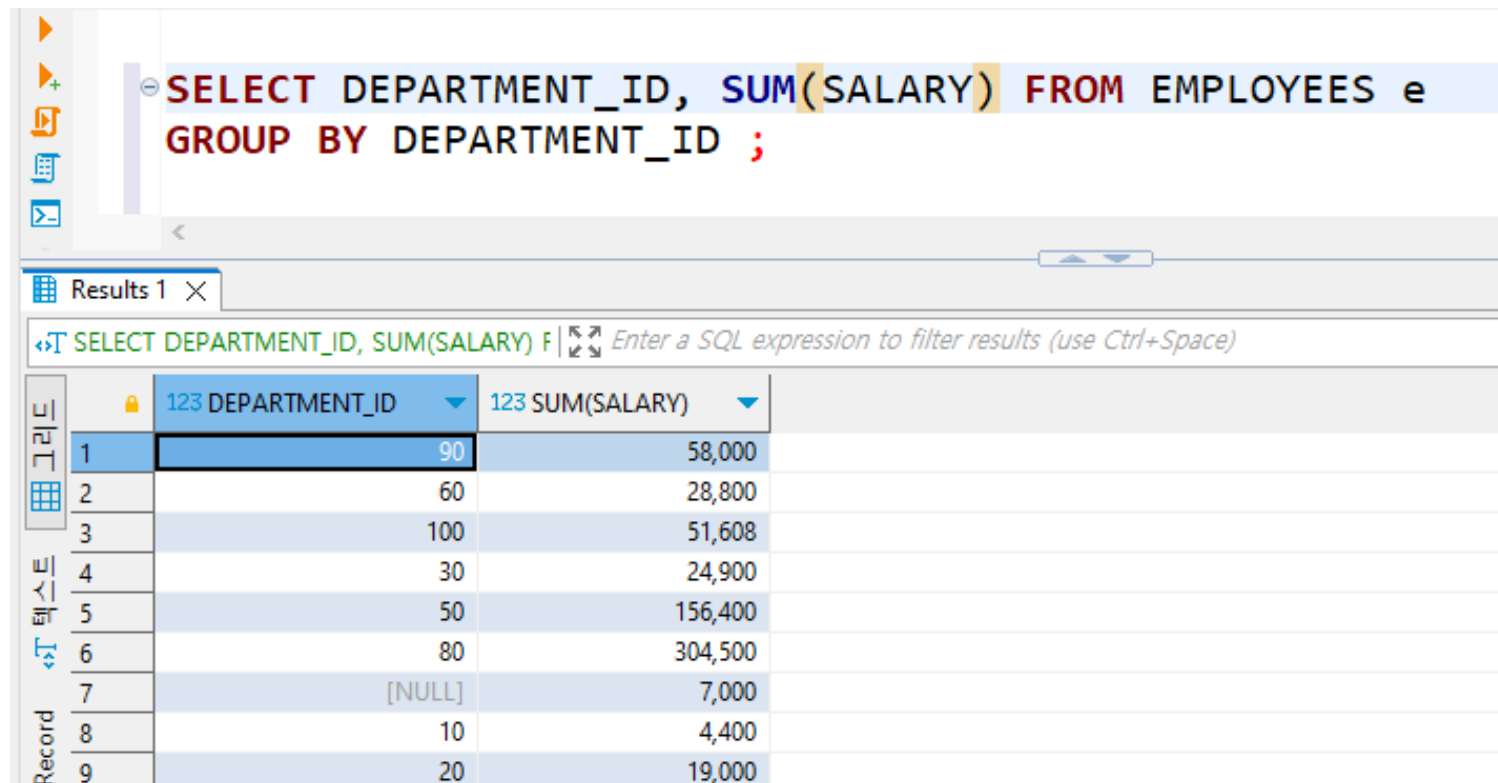
조회된 데이터의 여러 개 행을 특정 컬럼을 기준으로 묶어 그룹화 시키며
일반적으로 그룹 함수와 함께 사용됨

특정 컬럼을 기준으로 그룹화 하여 값을 조회할 수 있음

- WHERE을 사용한다면 GROUP BY 작성 이전에 사용 해야 함
- GROUP BY 이후 조건을 주고 싶다면 이후에 나오는 다른 방식으로 가능

- GROUP BY

EMPLOYEES 테이블에서 DEPARTMENT_ID를 기준으로 그룹화 한 뒤
각 부서별 합계 월급을 조회

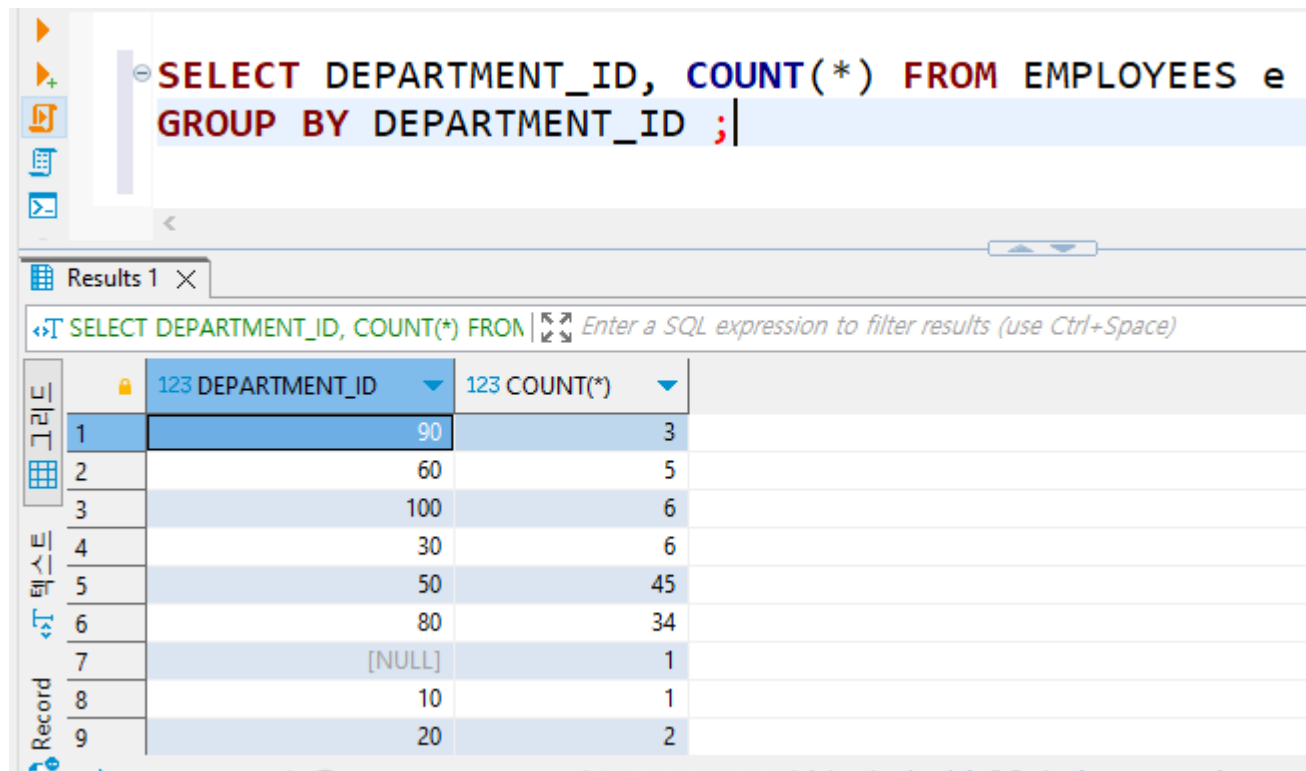


The screenshot shows a SQL IDE interface. At the top, a query is entered in a text editor: `SELECT DEPARTMENT_ID, SUM(SALARY) FROM EMPLOYEES e GROUP BY DEPARTMENT_ID ;`. Below the editor, the results are displayed in a table grid. The table has two columns: `DEPARTMENT_ID` and `SUM(SALARY)`. The results are grouped by department ID, with the first group (ID 90) having a sum of 58,000. The table grid shows 9 rows of data.

	DEPARTMENT_ID	SUM(SALARY)
1	90	58,000
2	60	28,800
3	100	51,608
4	30	24,900
5	50	156,400
6	80	304,500
7	[NULL]	7,000
8	10	4,400
9	20	19,000

- GROUP BY

EMPLOYEES 테이블에서 DEPARTMENT_ID를 기준으로 그룹화 한 뒤
각 부서별 존재하는 데이터의 행의 개수를 조회(즉, 부서별 직원 수 조회)



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

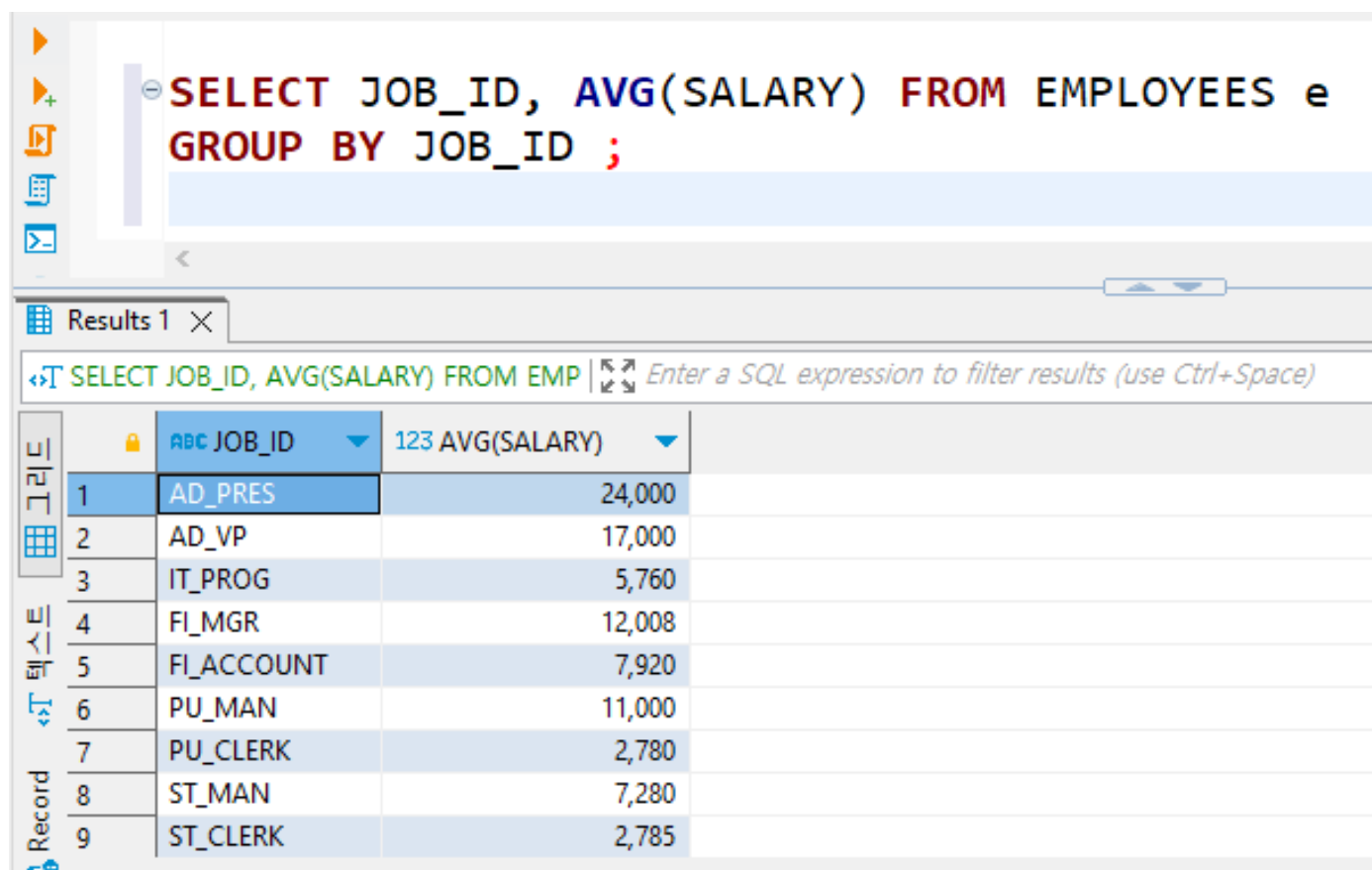
```
SELECT DEPARTMENT_ID, COUNT(*) FROM EMPLOYEES e  
GROUP BY DEPARTMENT_ID ;
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has two columns: "DEPARTMENT_ID" and "COUNT(*)". The results are as follows:

	DEPARTMENT_ID	COUNT(*)
1	90	3
2	60	5
3	100	6
4	30	6
5	50	45
6	80	34
7	[NULL]	1
8	10	1
9	20	2

- GROUP BY

JOB_ID가 일치하는 직원들의 평균 월급 조회



The screenshot shows a SQL IDE interface. At the top, a query editor contains the following SQL statement:

```
SELECT JOB_ID, AVG(SALARY) FROM EMPLOYEES e  
GROUP BY JOB_ID ;
```

Below the query editor, a results pane titled "Results 1" displays the output of the query. The results are shown in a table with two columns: "JOB_ID" and "AVG(SALARY)". The table contains 9 rows of data, representing different job positions and their average salaries.

	JOB_ID	AVG(SALARY)
1	AD_PRES	24,000
2	AD_VP	17,000
3	IT_PROG	5,760
4	FI_MGR	12,008
5	FI_ACCOUNT	7,920
6	PU_MAN	11,000
7	PU_CLERK	2,780
8	ST_MAN	7,280
9	ST_CLERK	2,785

• Q2. 실습

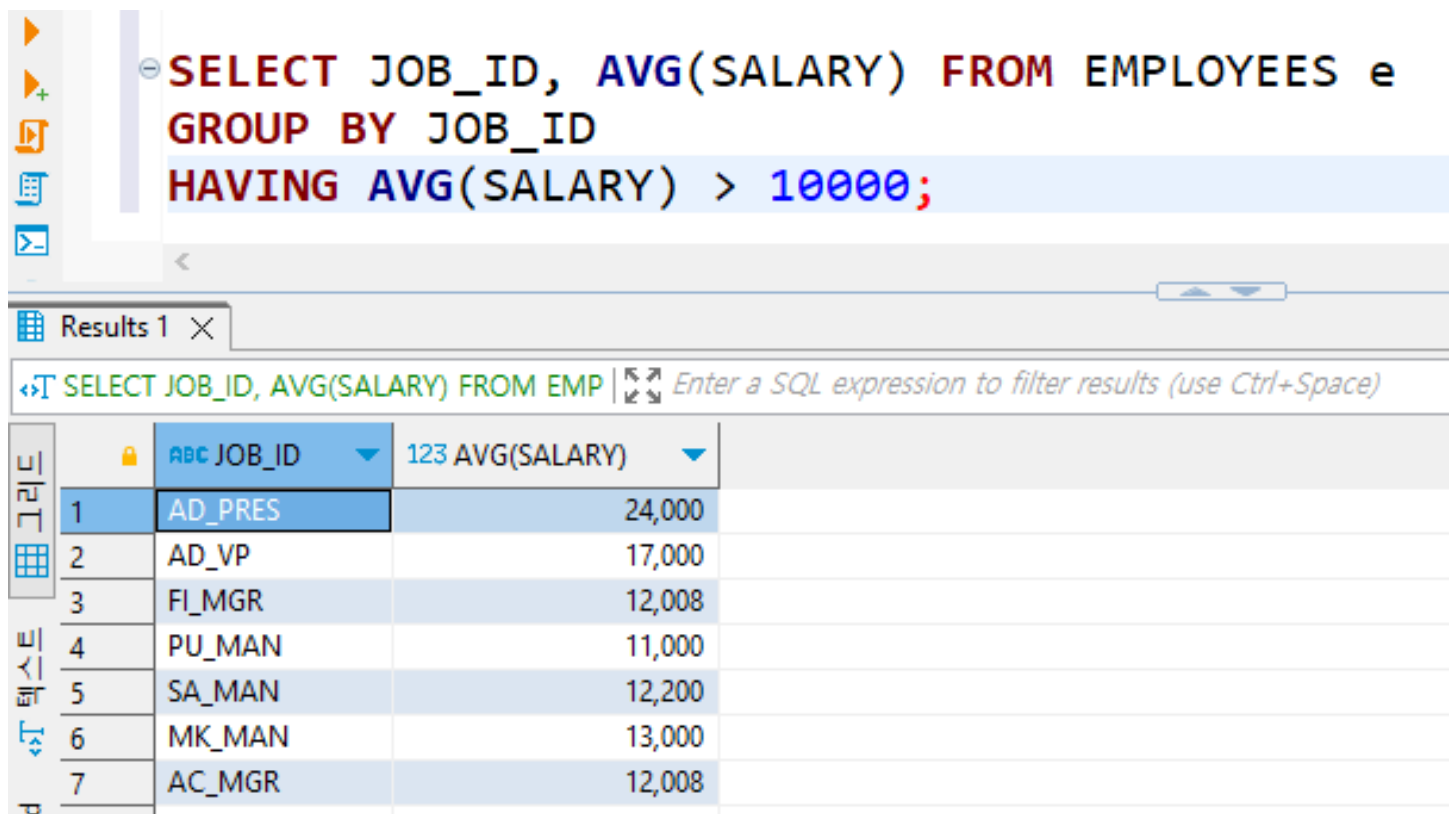
1. 직원들 중 DEPARTMENT_ID를 기준으로 아래의 요구사항에 맞게 데이터를 조회하세요.
 - 총 급여의 합
 - 사원 수
 - 보너스를 받는 사원 수
 - 최소 월급
 - 평균 월급
 - 최대 월급
 - 내림차순을 이용하여 데이터 출력
 - 각각의 값들을 모두 조회하고, 별칭을 사용하세요.

- **HAVING**

GROUP BY에서 조건을 제시할 때 사용되는 구문

- HAVING

JOB_ID를 기준으로 평균 월급을 구하되 10,000달러 이상인 직원들만 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

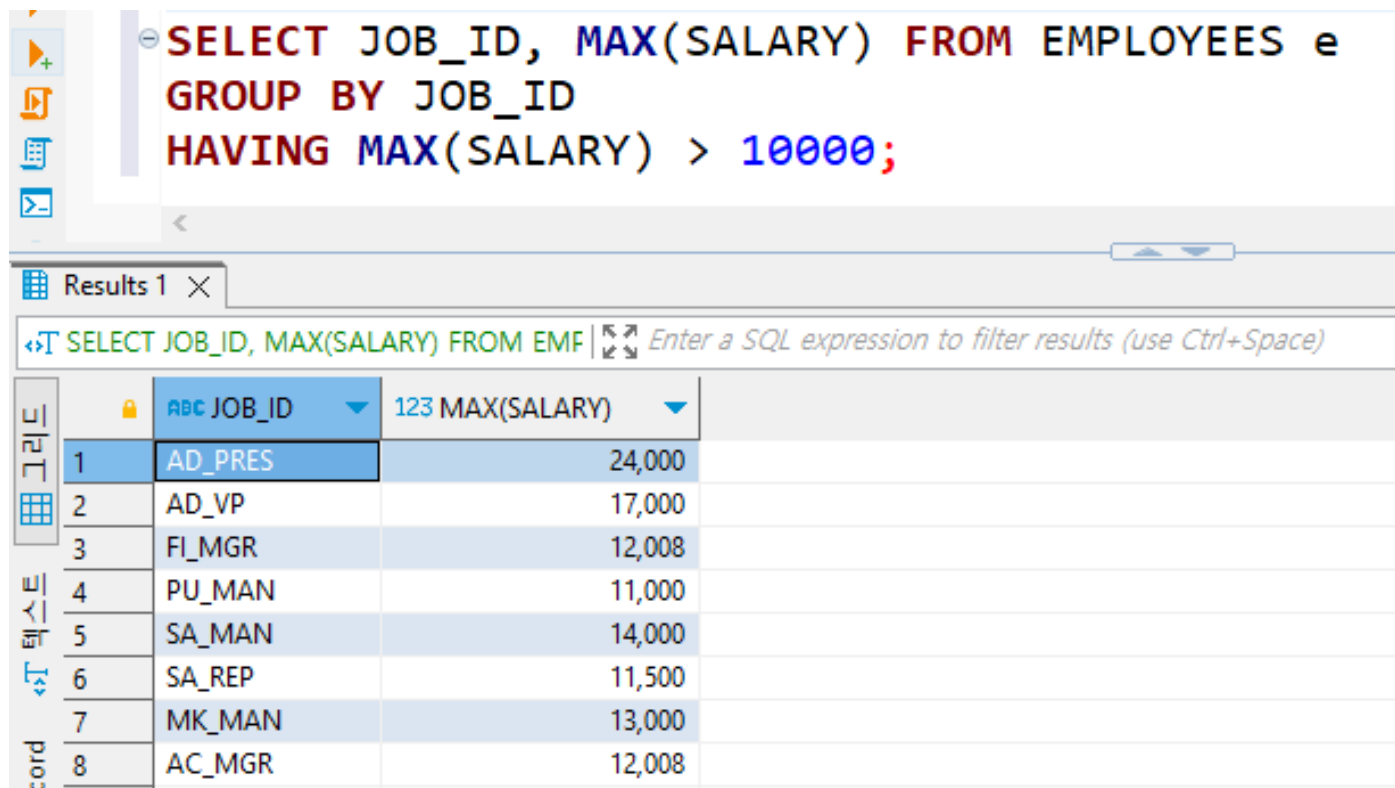
```
SELECT JOB_ID, AVG(SALARY) FROM EMPLOYEES e  
GROUP BY JOB_ID  
HAVING AVG(SALARY) > 10000;
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has two columns: "JOB_ID" and "AVG(SALARY)". The results are as follows:

JOB_ID	AVG(SALARY)
AD_PRES	24,000
AD_VP	17,000
FI_MGR	12,008
PU_MAN	11,000
SA_MAN	12,200
MK_MAN	13,000
AC_MGR	12,008

- HAVING

JOB_ID를 기준으로 가장 높은 월급을 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

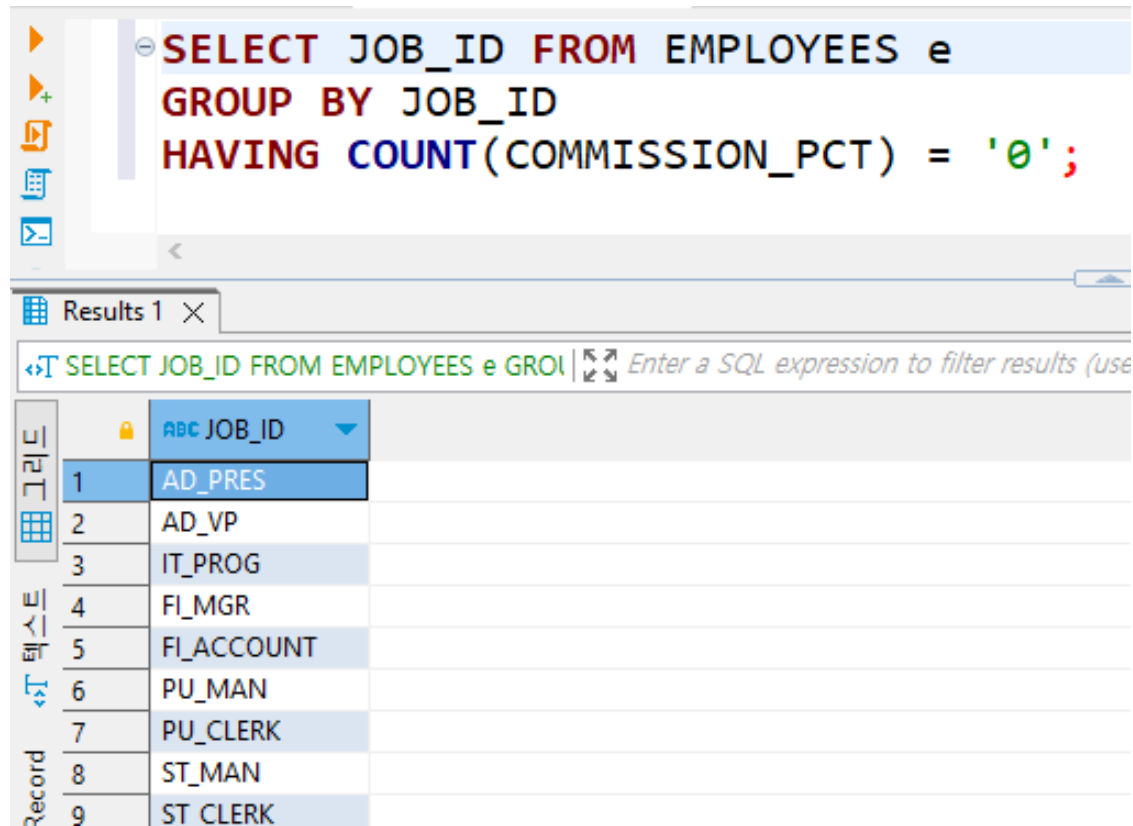
```
SELECT JOB_ID, MAX(SALARY) FROM EMPLOYEES e  
GROUP BY JOB_ID  
HAVING MAX(SALARY) > 10000;
```

The bottom pane, titled 'Results 1', displays the query results in a table. The table has two columns: 'JOB_ID' and 'MAX(SALARY)'. The results are as follows:

	JOB_ID	MAX(SALARY)
1	AD_PRES	24,000
2	AD_VP	17,000
3	FI_MGR	12,008
4	PU_MAN	11,000
5	SA_MAN	14,000
6	SA_REP	11,500
7	MK_MAN	13,000
8	AC_MGR	12,008

- HAVING

보너스를 받지 않는 JOB_ID 조회



The screenshot shows a SQL IDE interface. The top pane contains the following SQL query:

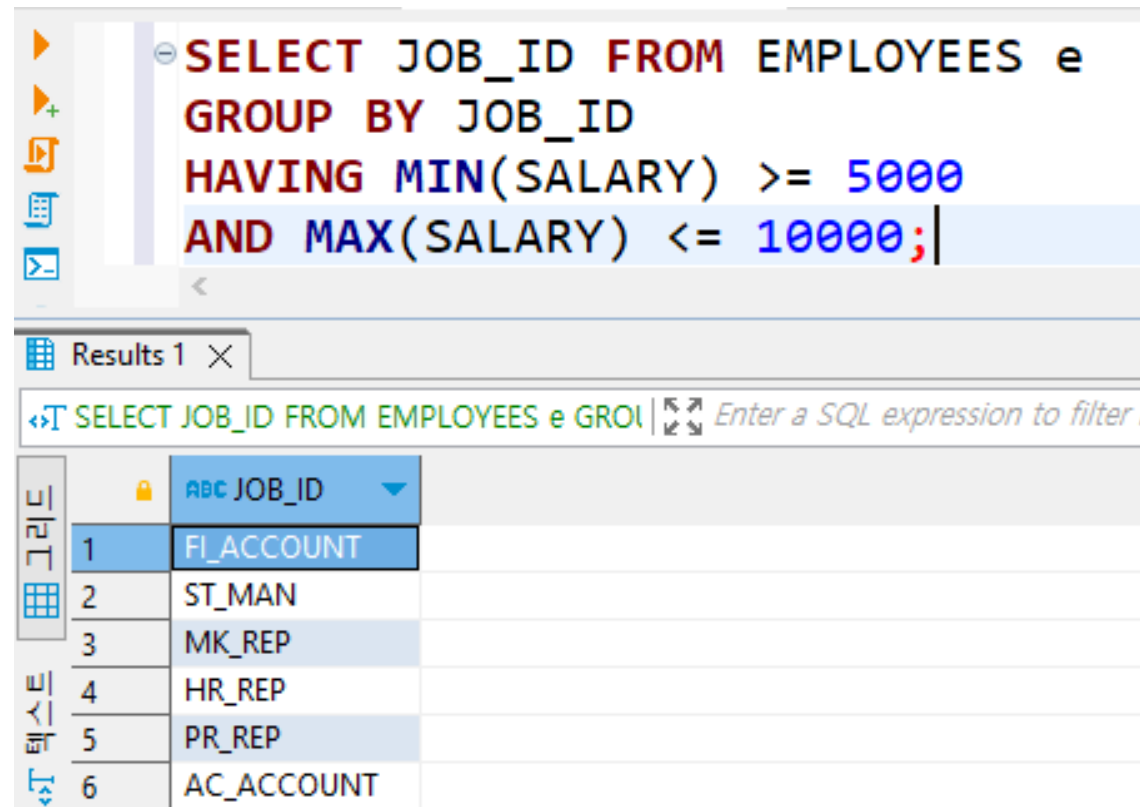
```
SELECT JOB_ID FROM EMPLOYEES e  
GROUP BY JOB_ID  
HAVING COUNT(COMMISSION_PCT) = '0';
```

The bottom pane, titled "Results 1", displays the query results in a table. The table has two columns: "JOB_ID" and "COUNT(COMMISSION_PCT)". The results are as follows:

JOB_ID	COUNT(COMMISSION_PCT)
AD_PRES	1
AD_VP	2
IT_PROG	3
FI_MGR	4
FI_ACCOUNT	5
PU_MAN	6
PU_CLERK	7
ST_MAN	8
ST_CLERK	9

- HAVING

가장 낮은 월급이 5,000 이상이면서 가장 높은 월급이 10,000 이하인 직원이 존재하는 JOB_ID 조회



The screenshot shows a SQL IDE interface. The top pane contains a SQL query: `SELECT JOB_ID FROM EMPLOYEES e GROUP BY JOB_ID HAVING MIN(SALARY) >= 5000 AND MAX(SALARY) <= 10000;`. The bottom pane, titled 'Results 1', displays the query results in a table with 6 rows. The first row is the header 'JOB_ID'. The subsequent rows are 'FI_ACCOUNT', 'ST_MAN', 'MK_REP', 'HR_REP', 'PR_REP', and 'AC_ACCOUNT'.

	JOB_ID
1	FI_ACCOUNT
2	ST_MAN
3	MK_REP
4	HR_REP
5	PR_REP
6	AC_ACCOUNT

• Q2. 실습

1. 직원들 중 DEPARTMENT_ID를 기준으로 아래의 요구사항에 맞게 데이터를 조회하세요.
 - 총 급여의 합
 - 사원 수
 - 보너스를 받는 사원 수
 - 최소 월급
 - 평균 월급
 - 최대 월급
 - 내림차순을 이용하여 데이터 출력
 - 각각의 값들을 모두 조회하고, 별칭을 사용하세요.
2. 위의 데이터에서 최소 월급이 1000 이고 최대 월급이 5000 이하인 직원을 조회하세요.

• Q2. 실습

1. 직원들 중 보너스를 받는 부서 번호(DEPARTMENT_ID)를 조회 하세요.
2. 보너스를 받으면서 월급의 합이 8,000 이상인 직원을 조회 하세요.
3. DEPARTMENT_ID를 가지고 있지 않는 직원을 조회 하세요.
4. 보너스를 포함한 월급이 3,000 이상인 직원들이 있는 JOB_ID를 조회하세요.
5. 가장 높은 평균 월급을 가지고 있는 상위 5개의 DEPARTMENT_ID를 조회하세요.

• 참고

쿼리의 마지막에 `FETCH FIRST 5 ROWS ONLY;`를 작성하면 상위 5개의 결과만을 조회할 수 있습니다.

- **SELECT 쿼리의 실행 순서 - 1**

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

- **실행 순서 예시 쿼리**

```
⇒ SELECT JOB_ID, MIN(SALARY) FROM EMPLOYEES e  
WHERE SALARY < 10000  
GROUP BY JOB_ID  
HAVING MIN(SALARY) >= 5000  
ORDER BY JOB_ID DESC;
```


- SELECT 쿼리의 실행 순서 - 2

1. FORM

	▼ ABC EMAIL ▼	▼ ABC PHONE_NUMBER ▼	🕒 HIRE_DATE ▼	▼ ABC JOB_ID ▼	▼ 123 SALARY ▼	▼ 123 COMMISSION_PCT ▼	▼ 123 MA
1	SKING	515.123.4567	2003-06-17 00:00:00.000	🔗 AD_PRES	24,000	[NULL]	
2	NKOCHHAR	515.123.4568	2005-09-21 00:00:00.000	🔗 AD_VP	17,000	[NULL]	
3	LDEHAAN	515.123.4569	2001-01-13 00:00:00.000	🔗 AD_VP	17,000	[NULL]	
4	AHUNOLD	590.423.4567	2006-01-03 00:00:00.000	🔗 IT_PROG	9,000	[NULL]	
5	BERNST	590.423.4568	2007-05-21 00:00:00.000	🔗 IT_PROG	6,000	[NULL]	
6	DAUSTIN	590.423.4569	2005-06-25 00:00:00.000	🔗 IT_PROG	4,800	[NULL]	
7	VPATABAL	590.423.4560	2006-02-05 00:00:00.000	🔗 IT_PROG	4,800	[NULL]	
8	DLORENTZ	590.423.5567	2007-02-07 00:00:00.000	🔗 IT_PROG	4,200	[NULL]	
9	NGREENBE	515.124.4569	2002-08-17 00:00:00.000	🔗 FI_MGR	12,008	[NULL]	
10	DFAVIET	515.124.4169	2002-08-16 00:00:00.000	🔗 FI_ACCOUNT	9,000	[NULL]	
11	JCHEN	515.124.4269	2005-09-28 00:00:00.000	🔗 FI_ACCOUNT	8,200	[NULL]	
12	ISCIARRA	515.124.4369	2005-09-30 00:00:00.000	🔗 FI_ACCOUNT	7,700	[NULL]	
13	JMURMAN	515.124.4469	2006-03-07 00:00:00.000	🔗 FI_ACCOUNT	7,800	[NULL]	
14	LPOPP	515.124.4567	2007-12-07 00:00:00.000	🔗 FI_ACCOUNT	6,900	[NULL]	
15	DRAPHEAL	515.127.4561	2002-12-07 00:00:00.000	🔗 PU_MAN	11,000	[NULL]	
16	AKHOO	515.127.4562	2003-05-18 00:00:00.000	🔗 PU_CLERK	3,100	[NULL]	

- SELECT 쿼리의 실행 순서 - 2

2. WHERE

	▼	🕒 HIRE_DATE ▼	ABC JOB_ID ▼	123 SALARY ▼	123 COMMISSION_PCT ▼	123 MANAGER_ID ▼	123 DEPARTMENT_ID ▼
1		2006-01-03 00:00:00.000	🔗 IT_PROG	9,000	[NULL]	102 🔗	60 🔗
2		2007-05-21 00:00:00.000	🔗 IT_PROG	6,000	[NULL]	103 🔗	60 🔗
3		2005-06-25 00:00:00.000	🔗 IT_PROG	4,800	[NULL]	103 🔗	60 🔗
4		2006-02-05 00:00:00.000	🔗 IT_PROG	4,800	[NULL]	103 🔗	60 🔗
5		2007-02-07 00:00:00.000	🔗 IT_PROG	4,200	[NULL]	103 🔗	60 🔗
6		2002-08-16 00:00:00.000	🔗 FI_ACCOUNT	9,000	[NULL]	108 🔗	100 🔗
7		2005-09-28 00:00:00.000	🔗 FI_ACCOUNT	8,200	[NULL]	108 🔗	100 🔗
8		2005-09-30 00:00:00.000	🔗 FI_ACCOUNT	7,700	[NULL]	108 🔗	100 🔗
9		2006-03-07 00:00:00.000	🔗 FI_ACCOUNT	7,800	[NULL]	108 🔗	100 🔗
10		2007-12-07 00:00:00.000	🔗 FI_ACCOUNT	6,900	[NULL]	108 🔗	100 🔗
11		2003-05-18 00:00:00.000	🔗 PU_CLERK	3,100	[NULL]	114 🔗	30 🔗
12		2005-12-24 00:00:00.000	🔗 PU_CLERK	2,900	[NULL]	114 🔗	30 🔗
13		2005-07-24 00:00:00.000	🔗 PU_CLERK	2,800	[NULL]	114 🔗	30 🔗
14		2006-11-15 00:00:00.000	🔗 PU_CLERK	2,600	[NULL]	114 🔗	30 🔗
15		2007-08-10 00:00:00.000	🔗 PU_CLERK	2,500	[NULL]	114 🔗	30 🔗
16		2004-07-18 00:00:00.000	🔗 ST MAN	8,000	[NULL]	100 🔗	50 🔗

- **SELECT 쿼리의 실행 순서 - 3**

3. GROUP BY

→ GROUP BY에 작성한 컬럼을 기준으로 각각 그룹화하여 묶음

FI_ACCOUNT	6,900
PU_CLERK	2,500
ST_MAN	5,800
SA_REP	6,100

• SELECT 쿼리의 실행 순서 - 4

4. HAVING

→ 참고 : 위와 같이 HAVING은 연산이 수행되기 전 이미 GROUP BY와 HAVING 단계를 거치기 때문에 조건을 줄 때 WHERE에서 처리할 수 있는 조건은 WHERE에서 주게되면 불필요한 단계를 거치지 않을 수 있기 때문에 퍼포먼스를 좋게 만들 수 있음

FI_ACCOUNT	6,900
PU_CLERK	2,500
ST_MAN	5,800
SA_REP	6,100

← 조건에 맞지 않는 그룹 제거

- SELECT 쿼리의 실행 순서 - 5

5. SELECT

→ 조건에 충족된 그룹들 묶어 출력할 준비를 하는 단계

ABC JOB_ID ▼	123 MIN(SALARY) ▼
FI_ACCOUNT	6,900
ST_MAN	5,800
SA_REP	6,100
MK_REP	6,000
HR_REP	6,500
AC_ACCOUNT	8,300

• SELECT 쿼리의 실행 순서 - 6

6. ORDER BY

→ 출력할 데이터들을 정렬

ABC JOB_ID ▼	123 MIN(SALARY) ▼
ST_MAN	5,800
SA_REP	6,100
MK_REP	6,000
HR_REP	6,500
FI_ACCOUNT	6,900
AC_ACCOUNT	8,300